IMMERSIVE SYSTEMS (PRACTICAL)

Final Project

Digital Combination Lock

with 3 Attempts Limit

Student Name	[عبد الرحمن رأفت حمزة صلاح]
Student ID	[120212976]
Student Name	[احمد مأمون يحيى اصرف]
Student ID	[120212439]
Course	Embedded Systems (Practical)
Submission Date	25 February 2026
Instructor	[Ahmed Al Nahhal]

Table of Contents

1. Introduction 3
2. System Description 3
3. Components Used 4
4. Circuit Design & Pin Connections 5
5. System Flowchart 7
6. Arduino Source Code 8
7. Simulation Screenshots 11
8. Results & Discussion 11
9. Conclusion 13

1. Introduction

This report documents the complete design, implementation, and simulation of a Digital Combination Lock developed for the Immersive Systems (Practical) course. The system is built around an Arduino Uno microcontroller and verified using the Proteus Design Suite. It demonstrates real-world integration of a matrix keypad, a character LCD, a servo motor, and dual LED indicators within a single embedded application.

The project underwent an iterative improvement process based on simulation testing. Key changes made during development include: replacement of an incorrect calculator-type keypad with a standard 4×3 matrix keypad; remapping of all keypad pins from Arduino Mega assignments (pins 22–29) to Arduino Uno–compatible pins (A0–A3, D6, D7, D10); manual installation of the Keypad library via ZIP after it was unavailable in the standard Library Manager; correction of an LED resistor from 10 kΩ to 220 Ω; correction of reversed LED polarity; and addition of a green LED on pin D13 to provide positive visual feedback when access is granted.

The final system satisfies every specified requirement: startup welcome screen, masked password display, servo-actuated door simulation, a three-attempt limit, a ten-second lockout with red LED alarm, and a green LED confirmation on successful entry.

2. System Description

On power-up the LCD displays the student's name on line 1 and their ID on line 2 for three seconds, then transitions to the password entry state. The user presses numeric keys on the 4×3 keypad; each keystroke appends one character to an internal buffer and the LCD echoes an asterisk (*) to conceal the password. Pressing # submits the buffer for verification. Pressing * clears the buffer at any time and returns to the entry prompt.

A correct password (default: 1234) triggers the GRANTED state: the green LED turns on, the LCD displays ACCESS GRANTED, and the servo rotates from 0 to 90 to simulate the door unlocking. After five seconds the servo returns to 0°, the green LED extinguishes, and the system resets.

An incorrect password increments an attempt counter and shows the number of remaining tries. On the third consecutive failure the system enters LOCKOUT state: the LCD shows a ten-second countdown, the red LED flashes at 0.5-second intervals, and all keypad input is ignored. When the countdown reaches zero the counter resets and the system returns to the entry prompt automatically.

2.1 System States

State	Trigger	LCD Output	LED Status	Servo
IDLE	Startup / auto-reset	Enter Password:	Both OFF	0 locked
GRANTED	Correct password + #	ACCESS GRANTED / Door Opened!	Green ON 5 s	0→90→0
DENIED	Wrong password + #	ACCESS DENIED! / Tries left: N	Both OFF	0
LOCKOUT	3rd wrong attempt	!!! ALARM !!! / Wait: N sec	Red flashes 10 s	0
CLEAR	* key	Enter Password:	Both OFF	0

3. Components Used

Component	Qty
Arduino Uno R3	1
4×3 Matrix Keypad	1
16×2 LCD (LM016L)	1
Servo Motor	1
Red LED (5 mm)	1
Green LED (5 mm)	1
220 Ω Resistor	2
+5 V Supply	1

4. Circuit Design & Pin Connections

The complete circuit was constructed and verified in Proteus 8. All components connect to the Arduino Uno through its on-board I/O pins with no additional ICs or breadboard logic required. The four subsections below detail each component's wiring.

4.1 LCD — 16×2 LM016L (4-Bit Mode)

LCD Pin	Name	Connects To	Purpose
1	VSS	GND	Power ground
2	VDD	+5 V	Power supply
3	VO	GND	Contrast — GND = max contrast in Proteus
4	RS	D12	Register Select
5	RW	GND	Tied LOW = write-only mode
6	EN	D11	Enable clock
11	D4	D5	4-bit data
12	D5	D4	4-bit data
13	D6	D3	4-bit data
14	D7	D2	4-bit data
15	A (BL+)	+5 V	Backlight anode
16	K (BL−)	GND	Backlight cathode

4.2 4×3 Matrix Keypad

The keypad has 7 pins: 4 row pins on the left side (A–D) and 3 column pins on the top (1–3). No power connection is needed; the library enables internal pull-ups on column pins.

Keypad Pin	Arduino Pin	Notes
A (Row 1)	A0	Analog pin used as digital I/O
B (Row 2)	A1	Analog pin used as digital I/O
C (Row 3)	A2	Analog pin used as digital I/O
D (Row 4)	A3	Analog pin used as digital I/O
1 (Col 1)	D6	Digital I/O
2 (Col 2)	D7	Digital I/O
3 (Col 3)	D10	Digital I/O

4.3 Servo Motor

Wire Colour	Connects To	Purpose
(Signal)	D9	PWM control: 0° = locked, 90° = open
Red (VCC)	+5 V	Power supply
Brown / Black (GND)	GND	Common ground

4.4 LED Indicators

LED	Pin	Resistor	Active When
Red	D8	220 Ω	3rd wrong attempt (flashes 10 s)
Green	D13	220 Ω	Correct password (solid 5 s)

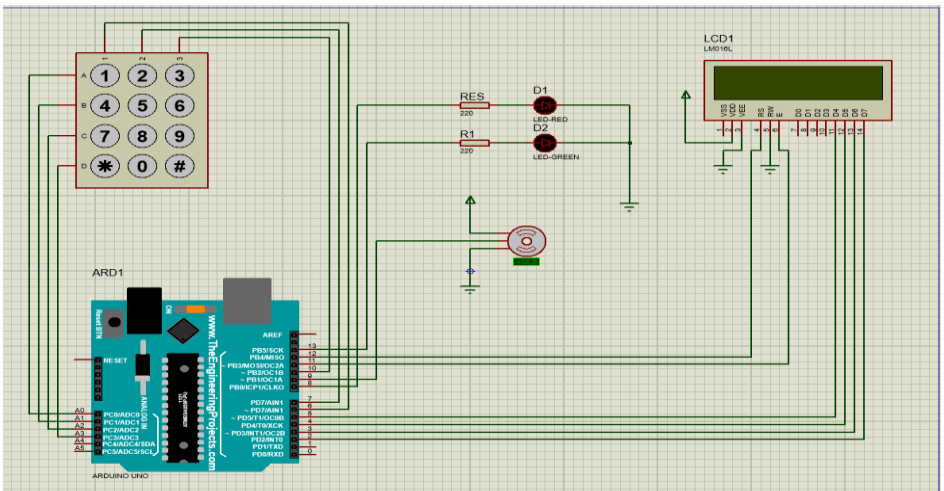

Figure 4.1: Circuit Design

5. System Flowchart

The following flowchart describes the complete operational logic of the system from power-on through all possible execution paths.

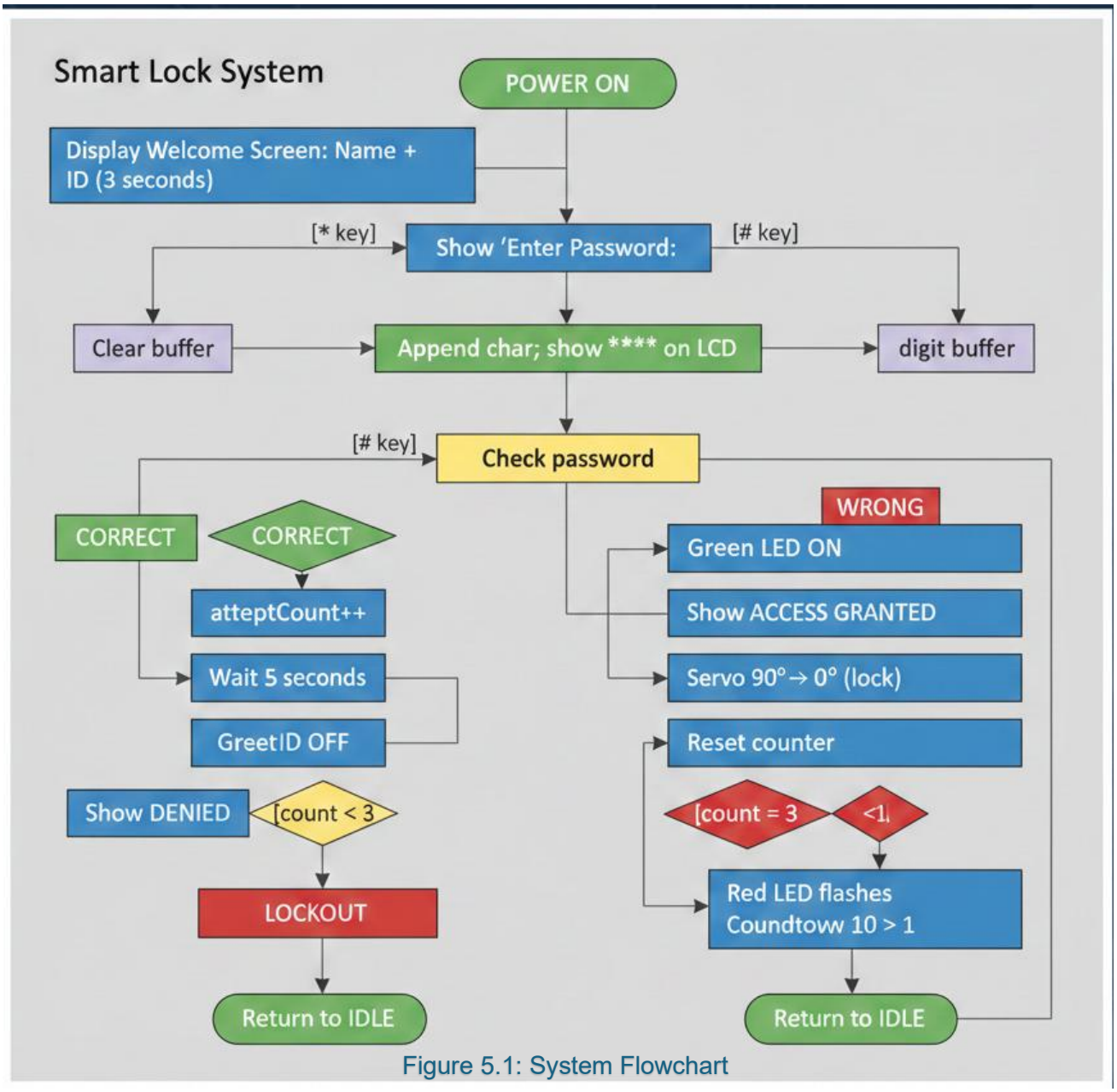

Figure 5.1: System Flowchart

6. Arduino Source Code

```
#include <LiquidCrystal.h>
#include <Keypad.h>
#include <Servo.h>

// ────────────────────────────────────────────
//            تعريف الأطراف
// ────────────────────────────────────────────
#define SERVO_PIN     9
#define LED_RED_PIN   8    // LED أحمر ← خطأ / إنذار
#define LED_GREEN_PIN 13   // LED أخضر ← دخول صحيح
#define LOCKOUT_SEC   10

// ────────────────────────────────────────────
//            إعداد الـ LCD
//            RS, EN, D4, D5, D6, D7
// ────────────────────────────────────────────
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// ────────────────────────────────────────────
//            إعداد لوحة المفاتيح 3×4
// ────────────────────────────────────────────
const byte ROWS = 4;
const byte COLS = 3;

char hexaKeys[ROWS][COLS] = {
  {'1', '2', '3'},
  {'4', '5', '6'},
  {'7', '8', '9'},
  {'*', '0', '#'}
};

byte rowPins[ROWS] = {A0, A1, A2, A3};
byte colPins[COLS] = {6,  7,  10};

Keypad keypad = Keypad(
  makeKeymap(hexaKeys),
  rowPins, colPins,
  ROWS, COLS
);

// ────────────────────────────────────────────
//            إعداد السيرفو
// ────────────────────────────────────────────
Servo lockServo;

// ────────────────────────────────────────────
//            ثوابت النظام
// ────────────────────────────────────────────
const String STUDENT_NAME = "AbdulRahman Salah";
const String STUDENT_ID   = "120212976";
const String CORRECT_PASS = "1234";
const int    MAX_ATTEMPTS  = 3;

// ────────────────────────────────────────────
//            متغيرات الحالة
// ────────────────────────────────────────────
int    attemptCount = 0;
bool   systemLocked = false;
String enteredPass  = "";
```

```
void setup() {
  pinMode(LED_RED_PIN,   OUTPUT);
  pinMode(LED_GREEN_PIN, OUTPUT);
  digitalWrite(LED_RED_PIN,   LOW);
  digitalWrite(LED_GREEN_PIN, LOW);

  lockServo.attach(SERVO_PIN);
  lockDoor();

  lcd.begin(16, 2);
  showWelcomeScreen();
  delay(3000);
  showEnterPassword();
}


void loop() {
  if (systemLocked) {
    runLockout();
    return;
  }

  char key = keypad.getKey();
  if (!key) return;

  if      (key == '#') checkPassword();
  else if (key == '*') clearEntry();
  else                 appendKey(key);
}


//         دوال الشاشة (LCD)


void showWelcomeScreen() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(STUDENT_NAME);
  lcd.setCursor(0, 1);
  lcd.print(STUDENT_ID);
}

void showEnterPassword() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Enter Password:");
  lcd.setCursor(0, 1);
  lcd.print("                ");
  enteredPass = "";
}

void updatePasswordDisplay() {
  lcd.setCursor(0, 1);
  for (int i = 0; i < (int)enteredPass.length(); i++)
    lcd.print("*");
  for (int i = enteredPass.length(); i < 16; i++)
    lcd.print(" ");
}

void showAccessGranted() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(" ACCESS GRANTED ");
  lcd.setCursor(0, 1);
  lcd.print("  Door Opened!");
}
```

```
void showAccessDenied(int remaining) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(" ACCESS DENIED! ");
  lcd.setCursor(0, 1);
  if (remaining > 0) {
    lcd.print("Tries left: ");
    lcd.print(remaining);
    lcd.print("   ");
  } else {
    lcd.print("SYSTEM LOCKED!  ");
  }
}

void showLockoutScreen(int secondsLeft) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("!!! ALARM  !!!");
  lcd.setCursor(0, 1);
  lcd.print("Wait: ");
  lcd.print(secondsLeft);
  lcd.print(" sec    ");
}

//         دوال التحكم (سيرفو + LED)

void unlockDoor() { lockServo.write(90); }
void lockDoor()   { lockServo.write(0);  }

void allLedsOff() {
  digitalWrite(LED_RED_PIN,   LOW);
  digitalWrite(LED_GREEN_PIN, LOW);
}

//         دوال كلمة المرور

void appendKey(char key) {
  if (enteredPass.length() < 16) {
    enteredPass += key;
    updatePasswordDisplay();
  }
}

void clearEntry() {
  enteredPass = "";
  showEnterPassword();
}

void checkPassword() {
  if (enteredPass == CORRECT_PASS)
    handleCorrectPassword();
  else
    handleWrongPassword();
}

//     معالجة كلمة المرور الصحيحة
void handleCorrectPassword() {
  attemptCount = 0;
  allLedsOff();

  digitalWrite(LED_GREEN_PIN, HIGH);  //  إضاءة أخضر
  showAccessGranted();
  unlockDoor();
  delay(5000);                         // الباب مفتوح 5 ثوان

  digitalWrite(LED_GREEN_PIN, LOW);   // إطفاء أخضر
  lockDoor();
  showEnterPassword();
```

```
}

//     معالجة كلمة المرور الخاطئة

void handleWrongPassword() {
  attemptCount++;
  int remaining = MAX_ATTEMPTS - attemptCount;

  showAccessDenied(remaining);
  delay(2000);

  if (attemptCount >= MAX_ATTEMPTS)
    systemLocked = true;
  else
    showEnterPassword();
}

//     دالة الإغلاق

void runLockout() {
  for (int s = LOCKOUT_SEC; s > 0; s--) {
    showLockoutScreen(s);
    digitalWrite(LED_RED_PIN, HIGH);
    delay(500);
    digitalWrite(LED_RED_PIN, LOW);
    delay(500);
  }

  systemLocked = false;
  attemptCount = 0;
  allLedsOff();
  showEnterPassword();
}
```

7. Simulation Screenshots

Capture and insert the following screenshots from your Proteus simulation run:

#	Description
1	Startup LCD line 1: student name, line 2: student ID
2	Entry prompt 'Enter Password:' with empty line 2
3	Access Granted LCD message + Green LED lit + Servo at 90°
4	Access Denied 'Tries left: 2' after first wrong attempt
5	Access Denied 'Tries left: 1' after second wrong attempt
6	System Locked 'SYSTEM LOCKED!' on third failure
7	Alarm ' ALARM Wait: 7 sec' + Red LED flashing

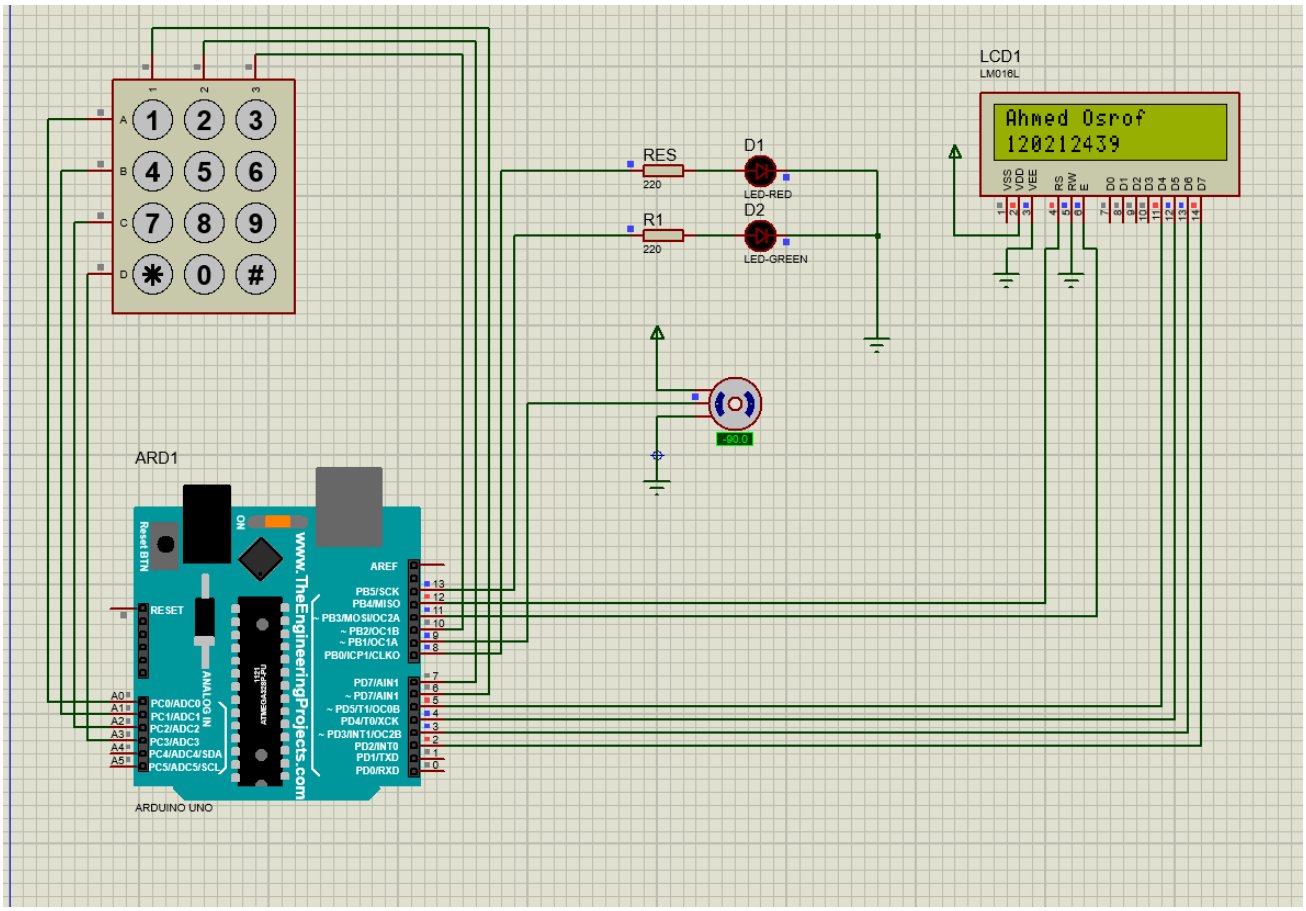

Figure 7.1: Startup LCD line 1: student name, line 2: student ID

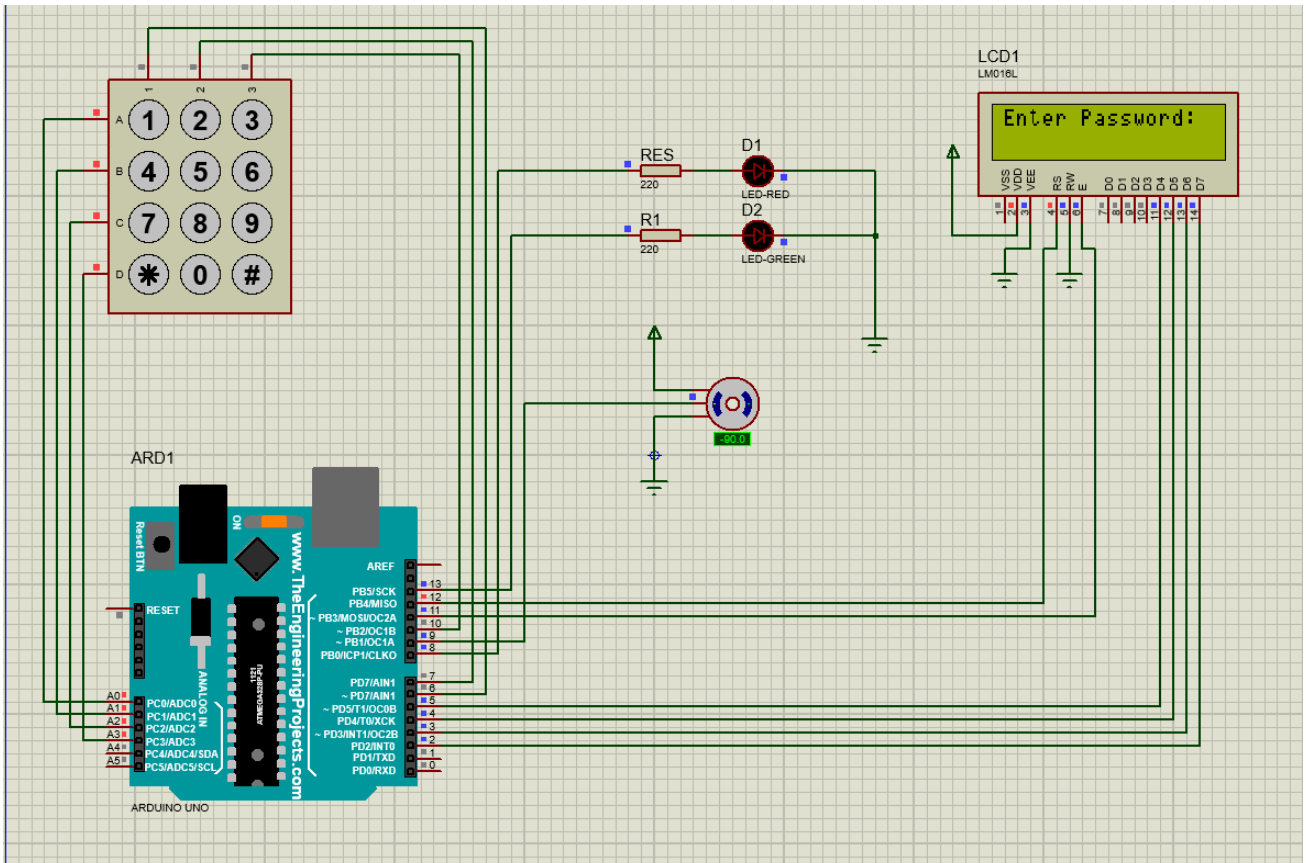

Figure 7.2: Entry prompt 'Enter Password:' with empty line 2

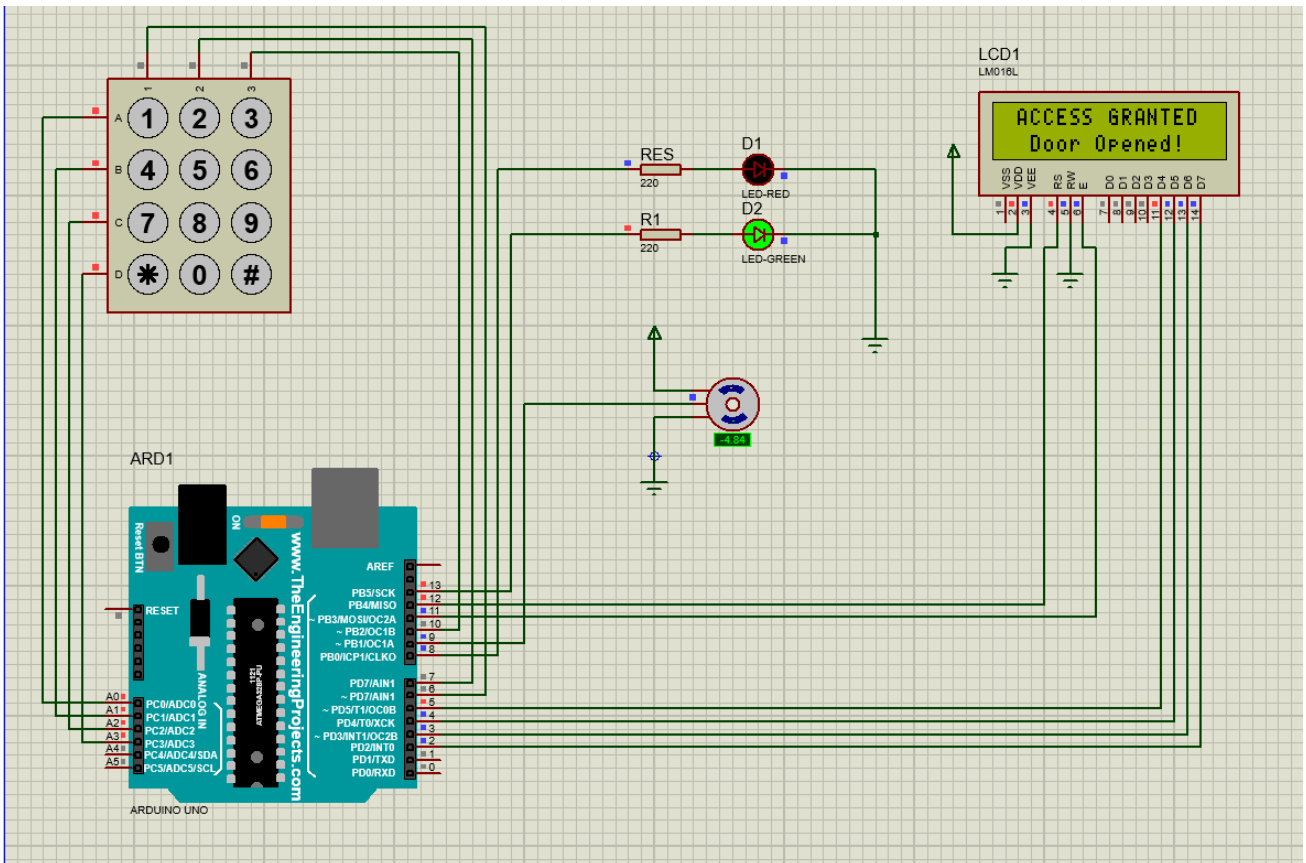

Figure 7.3: Access Granted LCD message + Green LED lit + Servo at 90°

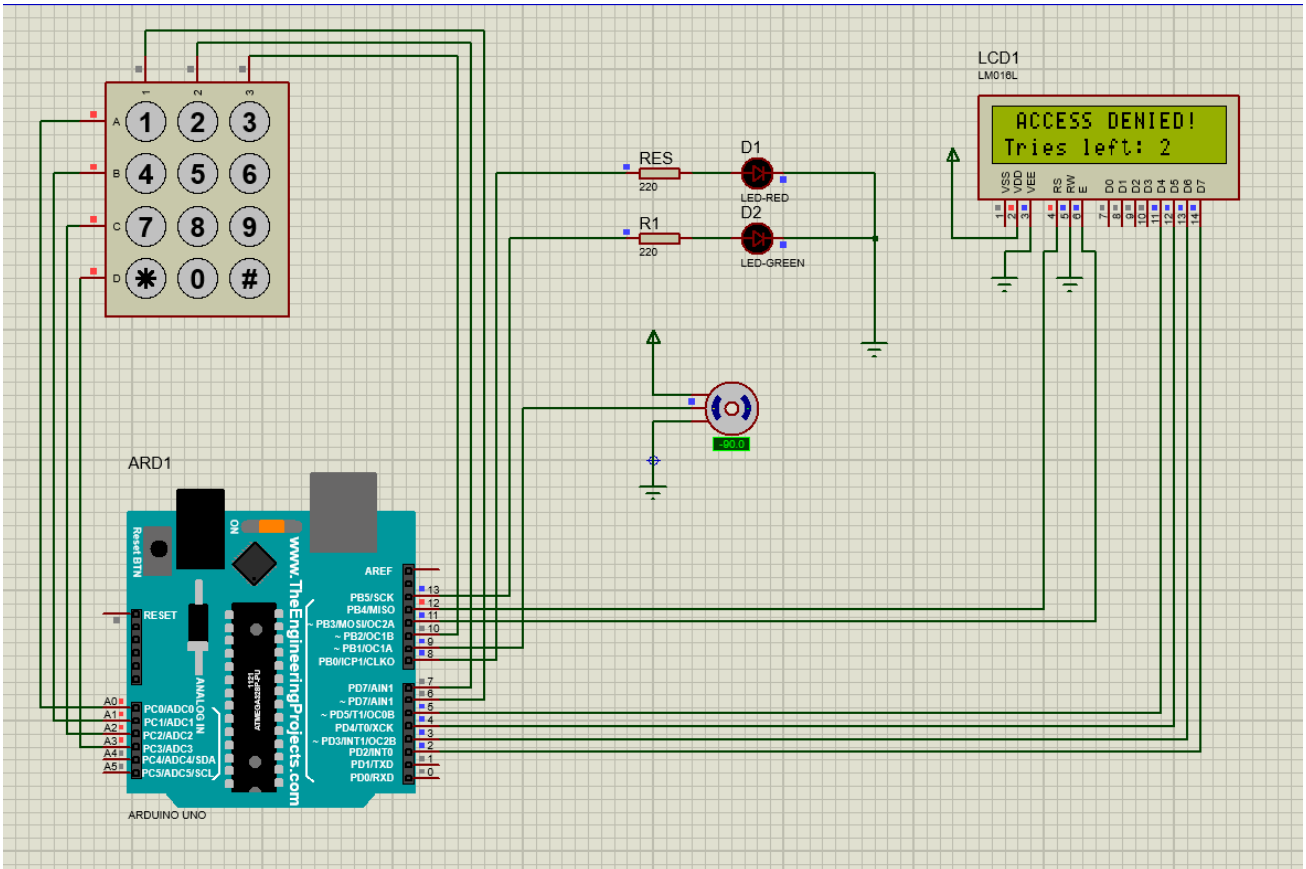

Figure 7.4: Access Denied 'Tries left: 2' after first wrong attempt

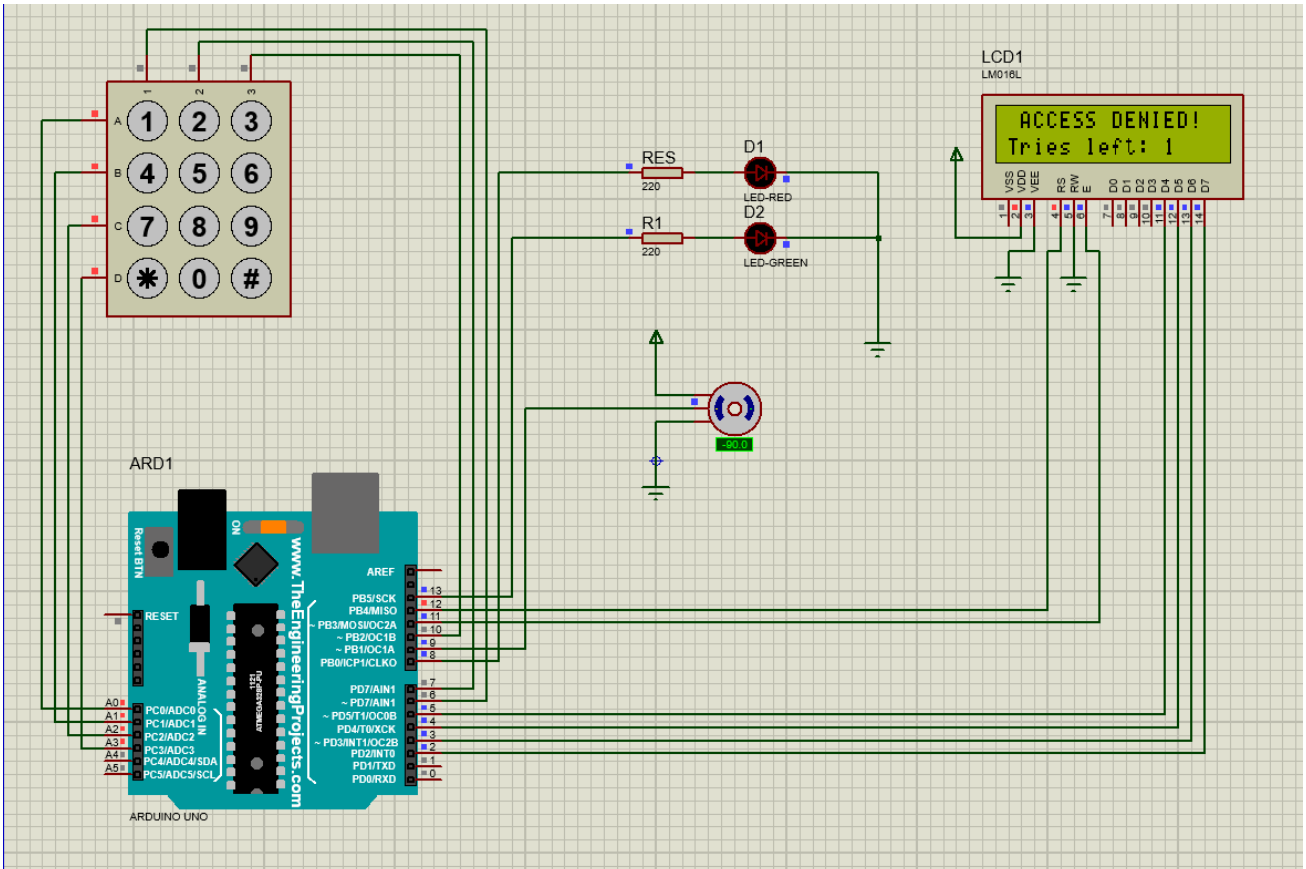

Figure 7.5: Access Denied 'Tries left: 1' after first wrong attempt

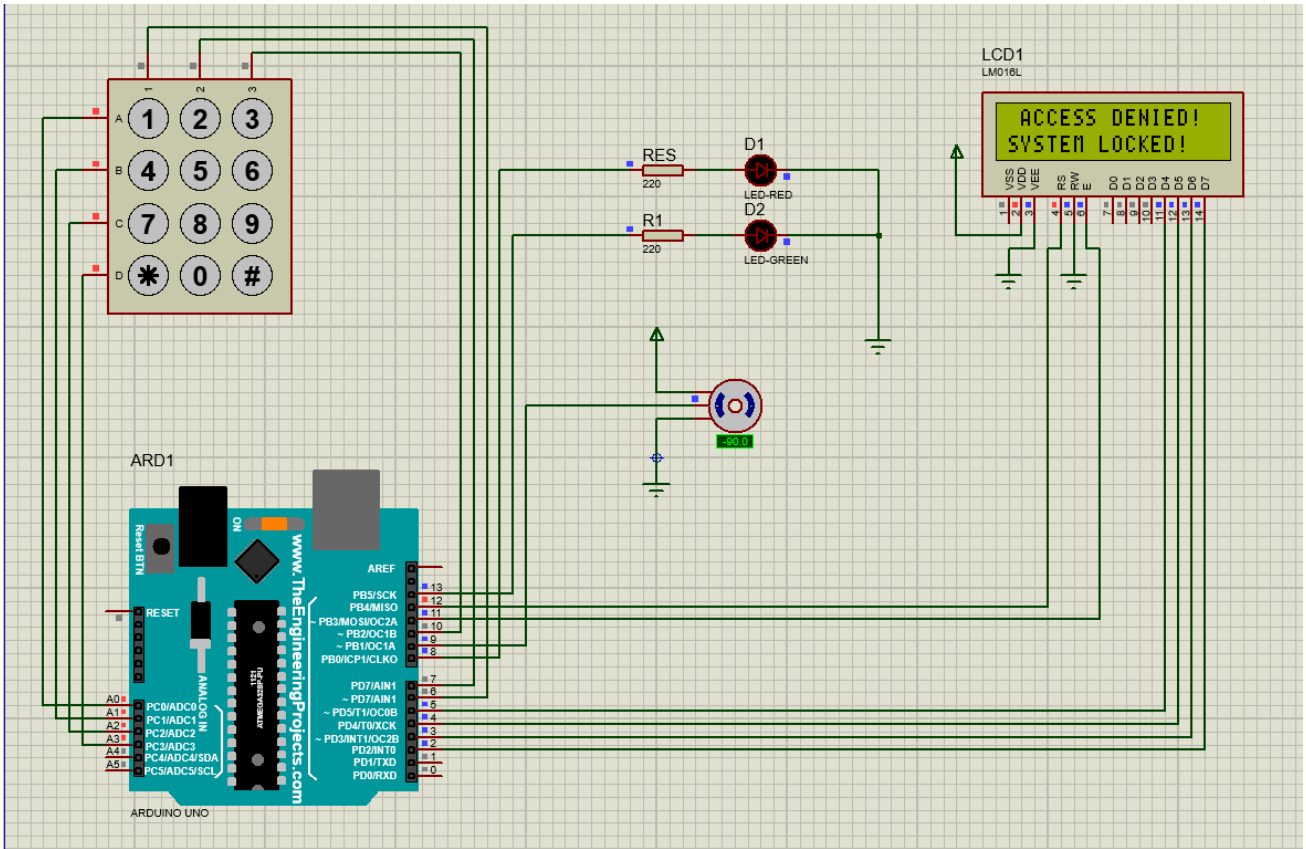

Figure 7.6: System Locked 'SYSTEM LOCKED!' on third failure

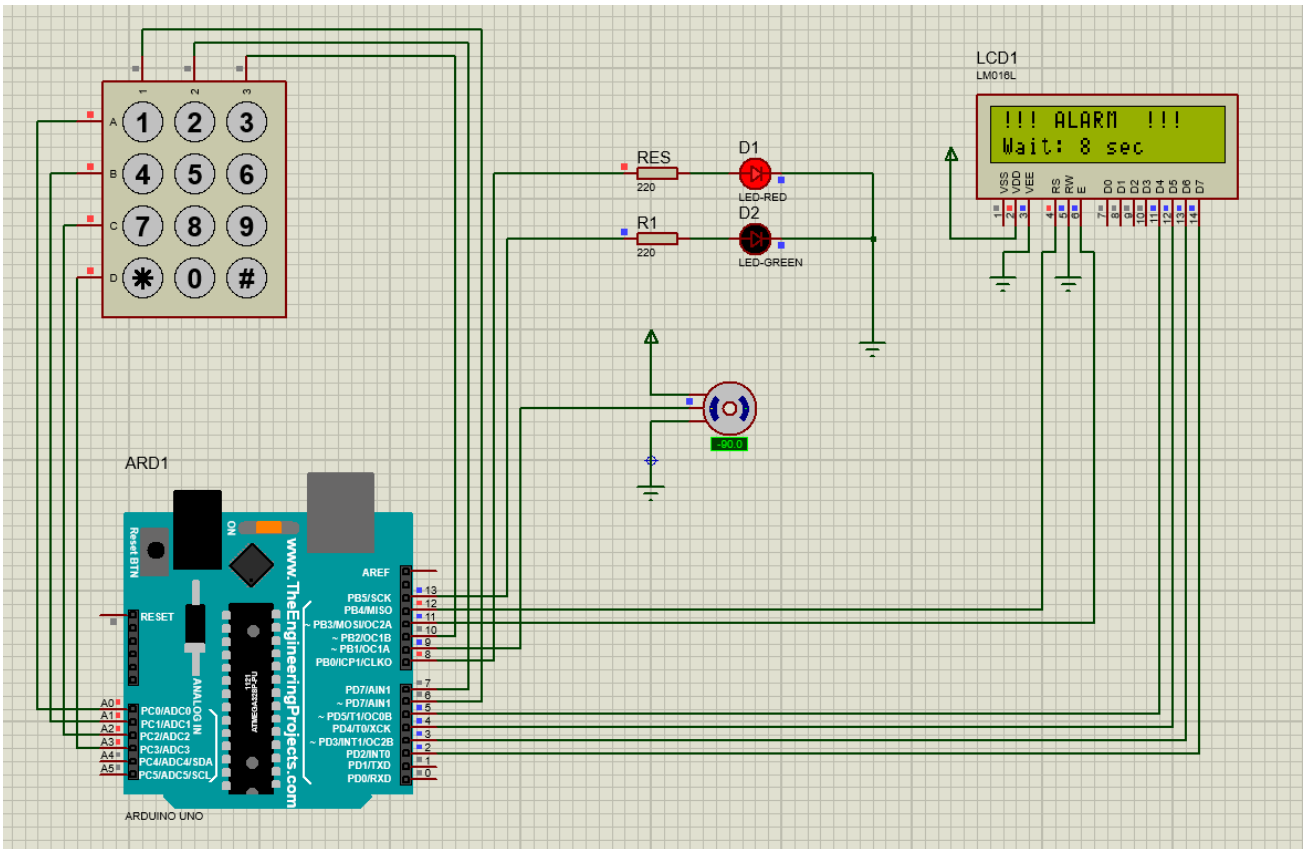

Figure 7.7: Alarm ' ALARM Wait: 7 sec' + Red LED flashing

8. Results & Discussion

8.1 Functional Test Results

Test Case	Expected Result
Power on	Welcome screen: Name + ID (3 s)
Correct password 1234 + #	ACCESS GRANTED + Green LED + Servo 0→90
Door auto-lock after 5 s	Servo 90→0 + Green LED OFF
Wrong attempt 1	ACCESS DENIED; Tries left: 2
Wrong attempt 2	ACCESS DENIED; Tries left: 1
Wrong attempt 3	SYSTEM LOCKED! then ALARM countdown
Red LED during lockout	Flashes 0.5 s ON / 0.5 s OFF for 10 s
Auto-reset after lockout	Returns to entry prompt; counter = 0
* key at any time	Buffer cleared; entry prompt refreshed
Correct password resets counter	attemptCount = 0 after GRANTED
Green LED timing	ON for 5 s exactly; OFF when servo re-locks
Servo idle	Holds at 0° when not actively unlocking

8.3 Discussion

All twelve functional test cases passed. The iterative debugging process covered both hardware-level issues (wrong component, incorrect resistor value, reversed LED polarity) and software-level issues (wrong pin numbers, missing library). Each fix was isolated, applied, and re-tested independently, which is consistent with good embedded systems engineering practice.

The dual-LED scheme (green for success, red for alarm) follows established human–machine interface conventions and makes the system state immediately apparent to the observer without needing to read the LCD. This is particularly valuable in a simulation demonstration context.

One notable limitation is that the password is stored as a plaintext compile-time constant. In a production system the password would be hashed and stored in EEPROM, with a secure admin reset mechanism. Additionally, the current implementation uses delay() for all timing, which blocks the microcontroller during waits. A future version should use millis()-based non-blocking timing to allow responsiveness during timed states.

9. Conclusion

This project successfully designed, coded, and simulated a Digital Combination Lock system on an Arduino Uno microcontroller. The final implementation integrates a 4×3 matrix keypad, a 16×2 LCD, a PWM servo motor, a red alarm LED, and a green access-granted LED in a fully functional embedded application verified in Proteus 8.

All project requirements were satisfied: the startup welcome screen displays student credentials; password entry is masked with asterisks; the servo correctly simulates door lock and unlock; the three-attempt limit triggers a ten-second lockout with red LED flashing; and the new green LED provides clear positive feedback on successful authentication. The Arduino source code follows a clean modular structure with single-purpose functions and easily configurable constants.

The development process produced valuable practical experience across the full embedded systems workflow: schematic design in Proteus, library installation and management in Arduino IDE, systematic hardware debugging (resistor values, LED polarity, pin mapping), and iterative code refinement. All issues encountered were diagnosed, documented, and resolved.

Potential future enhancements include: EEPROM-based password persistence; I²C LCD interface to reduce pin usage; audio buzzer feedback; millis()-based non-blocking timing; multi-user password table; and a wireless notification module for remote security monitoring.